



Institut National des Postes et Télécommunications
SUD — Systèmes Ubiquitaires et Distribués : Cloud & IoT



MyHeart Healthcare System

Nouvelles Fonctionnalités — Version 2.0

API Gateway · Circuit Breakers · Consul · Chatbot Multilingue

Informations du Projet

École :	INPT
Filière :	SUD
Étudiante :	Nasr Houda
Encadrant :	M. Hassan FARSI
Module :	Architecture Orientée Services (SOA)
Date :	March 7, 2026

Contents

1	Introduction	2
2	API Gateway	2
2.1	Description	2
2.2	Caractéristiques techniques	2
2.3	Routes configurées	3
2.4	Bénéfices	3
3	Circuit Breakers	4
3.1	Description	4
3.2	Principes de fonctionnement	4
3.3	Configuration	4
3.4	Mécanisme de fallback	4
3.5	Tests de résilience	5
3.6	Captures d'écran — Tests Opossum	5
4	Service Discovery avec Consul	7
4.1	Description	7
4.2	Services enregistrés	7
4.3	Interface Consul	7
4.4	Fonctionnement	7
4.5	Interface Consul — Capture d'écran	8
5	Chatbot Intelligent	9
5.1	Description	9
5.2	Contexte de développement — API IA évaluées	9
5.3	Solution implémentée	9
5.4	Réponses par catégorie	10
5.5	Adaptation par profil utilisateur	10
5.6	Évolution future	10
5.7	Capture d'écran — Interface Chatbot	11
6	Bilan des Tests	12
6.1	Tests de résilience	12
6.2	Métriques de performance	12
7	Difficultés Rencontrées	13
7.1	Problèmes techniques	13
7.2	Contraintes des API IA	13
8	Conclusion	14
8.1	Résumé des ajouts	14
8.2	Points forts	14
8.3	Points d'amélioration	14
8.4	Conclusion générale	14
8.5	Code source	14

1 Introduction

Ce rapport présente les nouvelles fonctionnalités ajoutées au projet MyHeart Healthcare System : une **API Gateway** centralisée, des **circuit breakers** pour la résilience, un serveur **Consul** pour la découverte des services Node.js, et un **chatbot multi-lingue** (français, anglais, arabe) adapté à chaque profil d'utilisateur. L'implémentation et les tests de ces ajouts y sont détaillés.

2 API Gateway

2.1 Description

Une **API Gateway** a été implémentée comme point d'entrée unique pour tous les microservices. Elle centralise toutes les requêtes provenant du frontend React et les route vers les services appropriés, assurant une gestion unifiée de la sécurité, du CORS et des logs.

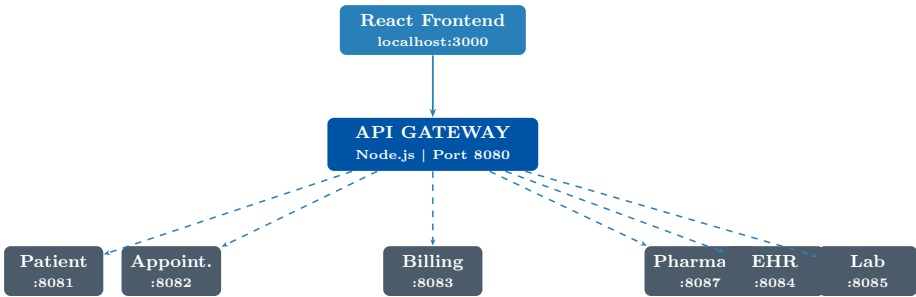


Figure 1: Architecture de l'API Gateway — point d'entrée unique vers les 6 microservices

2.2 Caractéristiques techniques

Paramètre	Valeur
Technologie	Node.js
Port d'écoute	8080
Services routés	6 microservices
Temps de réponse	< 50 ms

Table 1: Caractéristiques techniques de l'API Gateway

2.3 Routes configurées

Route	Service cible	Fonction
/api/auth	Patient Service	Authentification
/api/users	Patient Service	Gestion des utilisateurs
/api/patients	Patient Service	Gestion des patients
/api/appointments	Appointment Service	Rendez-vous
/api/bills	Billing Service	Facturation
/api/medications	Pharmacy Service	Médicaments
/api/prescriptions	Pharmacy Service	Prescriptions
/api/ehr	EHR Service	Dossiers médicaux
/api/lab	Lab Service	Laboratoire

Table 2: Routes configurées sur l'API Gateway

2.4 Bénéfices

- **Centralisation** : Un seul point d'entrée pour toute l'application
- **Sécurité** : Gestion unifiée des accès et du CORS
- **Monitoring** : Logs centralisés pour le suivi
- **Maintenance** : Configuration unique des routes

3 Circuit Breakers

3.1 Description

Des **circuit breakers** (disjoncteurs) ont été implémentés sur l’API Gateway pour protéger le système contre les pannes en cascade. Cette technique empêche qu’un service défaillant n’affecte l’ensemble de l’application en bloquant automatiquement les requêtes vers un service indisponible et en fournissant une réponse de secours.

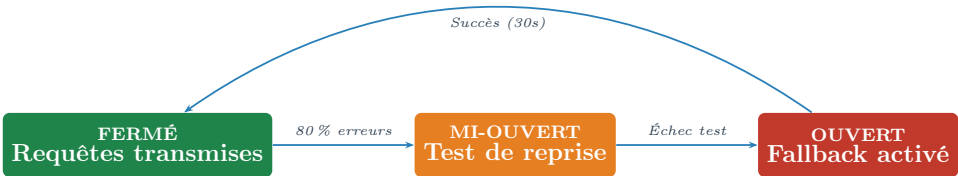


Figure 2: Machine d’états du Circuit Breaker : Fermé → Mi-ouvert → Ouvert

3.2 Principes de fonctionnement

État	Signification	Comportement
Fermé	Service opérationnel	Requêtes transmises normale-ment
Mi-ouvert	Test de reprise	Une requête test est autorisée
Ouvert	Service en panne	Requêtes bloquées, fallback ac-tivé

Table 3: Description des états du Circuit Breaker

3.3 Configuration

Paramètre	Valeur
Timeout	5 secondes
Seuil d’erreur	80 % d’échecs pour ouverture
Tentative de reprise	30 secondes
Fenêtre d’analyse	10 secondes

Table 4: Paramétrage du Circuit Breaker

3.4 Mécanisme de fallback

En cas d’indisponibilité d’un service, une réponse de secours est automatiquement envoyée à l’utilisateur :

```
1 "Service temporairement indisponible. Veuillez reessayer plus tard
  ."
```

Listing 1: Réponse de secours (fallback)

3.5 Tests de résilience

Scénario	Résultat
Service arrêté	[OK] Circuit s'ouvre après 80 % d'erreurs
Requêtes en arrêt	[OK] Fallback activé avec message approprié
Service rétabli	[OK] Circuit se referme automatiquement
Service instable	[OK] Protection efficace contre les pannes

Table 5: Résultats des tests de résilience

- **Résilience** : Protection contre les pannes en cascade
- **Disponibilité** : Réponses de secours maintenues
- **Auto-cicatrisation** : Reprise automatique après 30 secondes
- **Monitoring** : Suivi visuel des états en temps réel

3.6 Captures d'écran — Tests Opossum

Les captures ci-dessous illustrent les différents scénarios de test du circuit breaker Opossum en action sur l'API Gateway.

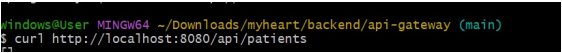


Figure 3: Test 1 — Test d'une API Circuit ouvert



Figure 4: Test 2 — Fallback activé + Circuit mi-ouvert (test de reprise)

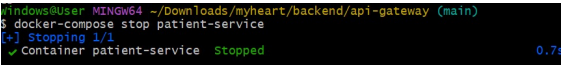


Figure 5: Test 3 — service arrêté

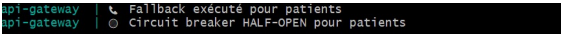


Figure 6: Test 4 — Circuit mi-ouvert

```
pi-gateway | ● Circuit breaker OPEN pour patients
pi-gateway | ☎ Fallback exécuté pour patients
pi-gateway | 172.21.0.1 - - [06/Mar/2026:18:17:31 +0000] "GET /api/patients HT
P/1.1" 503 111 "-" "curl/8.17.0"
pi-gateway | ☎ Fallback exécuté pour patients
pi-gateway | 172.21.0.1 - - [06/Mar/2026:18:17:32 +0000] "GET /api/patients HT
P/1.1" 503 111 "-" "curl/8.17.0"
pi-gateway | ☎ Fallback exécuté pour patients
pi-gateway | 172.21.0.1 - - [06/Mar/2026:18:17:33 +0000] "GET /api/patients HT
P/1.1" 503 111 "-" "curl/8.17.0"
pi-gateway | ☎ Fallback exécuté pour patients
pi-gateway | 172.21.0.1 - - [06/Mar/2026:18:17:34 +0000] "GET /api/patients HT
P/1.1" 503 111 "-" "curl/8.17.0"
pi-gateway | ☎ Fallback exécuté pour patients
pi-gateway | 172.21.0.1 - - [06/Mar/2026:18:17:35 +0000] "GET /api/patients HT
P/1.1" 503 111 "-" "curl/8.17.0"
```

Figure 7: Test 5 — Service ouvert

4 Service Discovery avec Consul

4.1 Description

Un serveur **Consul** a été déployé pour permettre la découverte automatique des services **Node.js**. Cette technologie permet aux services de s'enregistrer dynamiquement et d'être localisés sans configuration manuelle. Les services s'enregistrent au démarrage et se désenregistrent à l'arrêt automatiquement.



Figure 8: Consul Service Discovery — EHR Service & Lab Service (Node.js uniquement)

4.2 Services enregistrés

Service	Type	Port	Statut	Health Check
ehr-service	Node.js	8084	Enregistré	/health — 10s
lab-service	Node.js	8085	Enregistré	/health — 10s

Table 6: Services Node.js enregistrés dans Consul

```

windows@User MINGW64 ~/Downloads/myheart/frontend (main)
$ # Tester l'endpoint health de l'API Gateway
curl http://localhost:8080/health
{"status":"OK","service":"api-gateway","timestamp":"2026-03-07T19:54:18.255Z"}
windows@User MINGW64 ~/Downloads/myheart/frontend (main)
$

```

Figure 9: Test 6 — L'endpoint health de l'API Gateway

4.3 Interface Consul

L'interface web Consul est accessible à l'adresse <http://localhost:8500>. Elle offre une visualisation en temps réel des services enregistrés, le monitoring des health checks, la topologie du réseau et la gestion des pannes.

4.4 Fonctionnement

Les services Node.js s'enregistrent automatiquement au démarrage et se désenregistrent à l'arrêt. Des health checks sont effectués toutes les **10 secondes** sur l'endpoint `/health` de chaque service pour garantir leur disponibilité.

4.5 Interface Consul — Capture d'écran

La capture ci-dessous montre l'interface web Consul avec les services **ehr-service** et **lab-service** enregistrés et opérationnels.

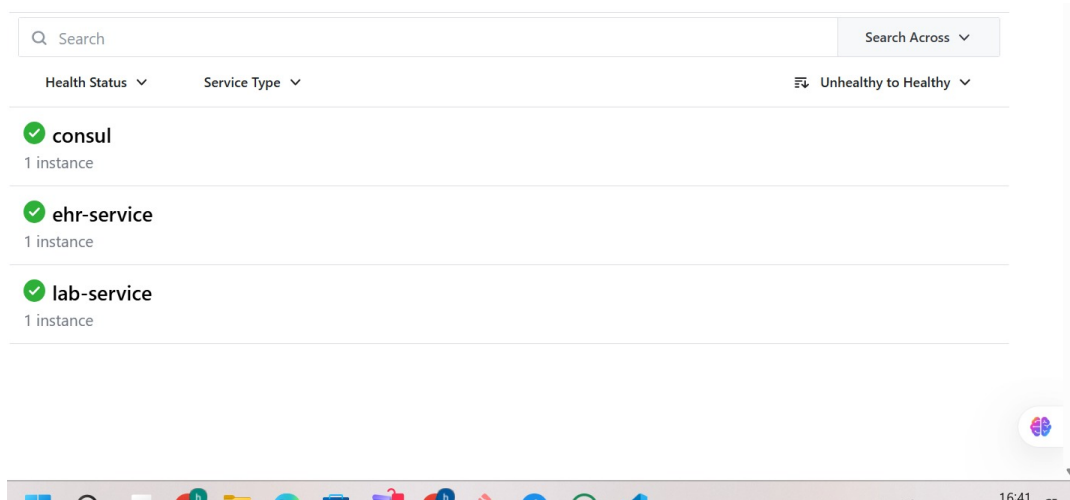


Figure 10: Interface Consul (<http://localhost:8500>) — EHR Service & Lab Service enregistrés

5 Chatbot Intelligent

5.1 Description

Un **assistant virtuel** a été intégré à l’application pour assister les utilisateurs selon leur profil. Le chatbot est accessible sur toutes les pages via un bouton flottant positionné en bas à droite de l’écran.

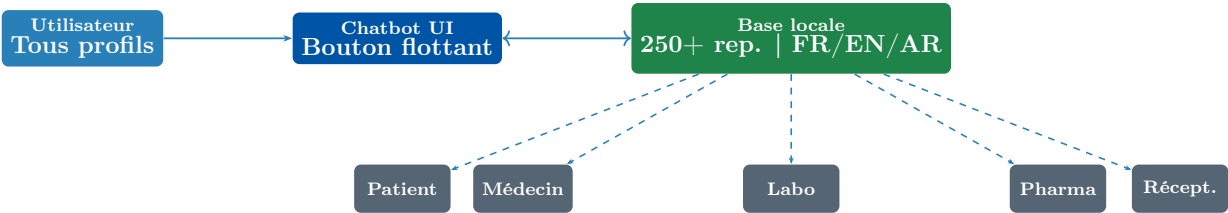


Figure 11: Architecture du Chatbot — base locale multilingue & adaptation par profil

5.2 Contexte de développement — API IA évaluées

API	Problème rencontré
OpenAI	Crédits gratuits limités
Google Gemini	Quota journalier trop restrictif
DeepSeek	Solde payant requis
OpenRouter	Modèles gratuits instables

Table 7: API IA évaluées et problèmes rencontrés

5.3 Solution implémentée

Face à ces contraintes, une version **démo** fonctionnelle du chatbot a été développée avec une base de connaissances locale dépassant 250 réponses, fonctionnant entièrement hors ligne avec un temps de réponse instantané.

Caractéristique	Valeur
Base de connaissances	250+ réponses locales
Mode de fonctionnement	100 % hors ligne
Temps de réponse	< 100 ms (instantané)
Langues supportées	Français · Anglais · Arabe

Table 8: Caractéristiques de la solution chatbot implémentée

5.4 Réponses par catégorie

Catégorie	Nb réponses	Exemples
Laboratoire	80+	Glycémie, cholestérol, CRP, TSH
Médicaments	30+	Paracétamol, ibuproène, antibiotiques
Pathologies	40+	Grippe, diabète, hypertension
Rendez-vous	15+	Prise, annulation, modification
Réception	80+	Inscription, accueil, planning
Anglais	30+	Traduction des questions courantes
Arabe	20+	Support des utilisateurs arabophones

Table 9: Distribution des réponses par catégorie

5.5 Adaptation par profil utilisateur

Profil	Assistance fournie
Patient	Rendez-vous, médicaments, factures
Médecin	Prescriptions, dossiers patients
Laboratoire	Analyses, normes, résultats
Pharmacie	Stocks, délivrances, génériques
Réception	Accueil, inscriptions, planning

Table 10: Assistance du chatbot selon le profil utilisateur

5.6 Évolution future

Le chatbot est conçu pour évoluer vers une vraie IA dès qu'une solution API gratuite et stable sera disponible, ou après acquisition de crédits payants. L'architecture modulaire permet une migration sans refonte du frontend.

5.7 Capture d'écran — Interface Chatbot

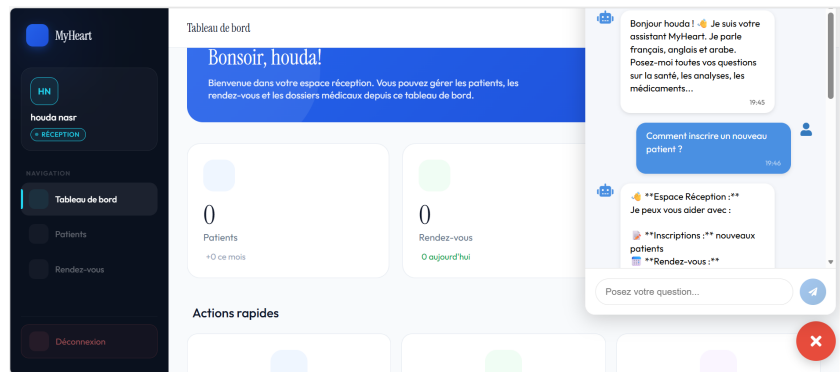


Figure 12: Interface du Chatbot — assistant virtuel multilingue (FR / EN / AR)

6 Bilan des Tests

6.1 Tests de résilience

Composant	Test effectué	Résultat
API Gateway	Routage vers tous les services	Opérationnel
Circuit Breakers	Arrêt/redémarrage de services	Fonctionnel
Consul	Enregistrement services Node.js	Opérationnel
Chatbot	Réponses en 3 langues	Fonctionnel

Table 11: Résultats des tests de résilience

6.2 Métriques de performance

Métrique	Valeur
Temps de réponse API Gateway	< 50 ms
Temps de réponse chatbot	< 100 ms
Services actifs simultanément	6/6
Taux de couverture des réponses	250+ questions
Langues supportées	3 (FR, EN, AR)

Table 12: Métriques de performance mesurées

- Tous les composants ont passé leurs tests de validation
- L’API Gateway route correctement vers les 6 microservices
- Les circuit breakers s’activent et se rétablissent automatiquement
- Le chatbot répond instantanément dans les 3 langues

7 Difficultés Rencontrées

7.1 Problèmes techniques

Difficulté	Solution adoptée
Intégration Consul avec Spring Boot	Reportée — focus sur les services Node.js
API IA gratuites indisponibles	Développement d'une version démo locale
Configuration des circuit breakers	Ajustement des paramètres par tests itératifs

Table 13: Difficultés rencontrées et solutions adoptées

7.2 Contraintes des API IA

Aucune solution d'API IA gratuite et stable n'a été trouvée pour une intégration en production.

⇒ **Décision** : développement d'une version démo fonctionnelle — preuve de concept valide en attendant l'accès à une vraie IA.

8 Conclusion

8.1 Résumé des ajouts

Domaine	Avant v2.0	Après v2.0
Architecture	Appels directs aux services	API Gateway centralisée
Résilience	Aucune protection	Circuit breakers actifs
Découverte	Configuration manuelle	Consul (Node.js)
Support	Aucun	Chatbot multilingue 24/7

Table 14: Bilan comparatif avant/après v2.0

8.2 Points forts

- + Architecture robuste avec protection contre les pannes
- + Expérience utilisateur améliorée grâce au chatbot
- + Support multilingue (français, anglais, arabe)
- + Monitoring via Consul et logs détaillés
- + Évolutivité de l'architecture pour la production

8.3 Points d'amélioration

- ! Extension de Consul aux services Spring Boot (migration Eureka)
- ! Intégration future d'une vraie IA (GPT, Gemini ou modèle local)
- ! Tests de charge supplémentaires (JMeter, k6)

8.4 Conclusion générale

Les ajouts réalisés dans cette **version 2.0** apportent des améliorations significatives au système MyHeart. L'application est désormais plus **robuste**, **maintenable** et **facile à utiliser**, avec une architecture prête pour l'évolution vers une solution de production cloud-native.

8.5 Code source

Le code source complet du projet est disponible sur GitHub :

Dépôt GitHub :

<https://github.com/NASRHOUDA/myheart-healthcare-system.git>